ROCmPort AI

CUDA-to-ROCm Migration powered by Qwen

Built for the AMD Developer Hackathon

1. The Problem

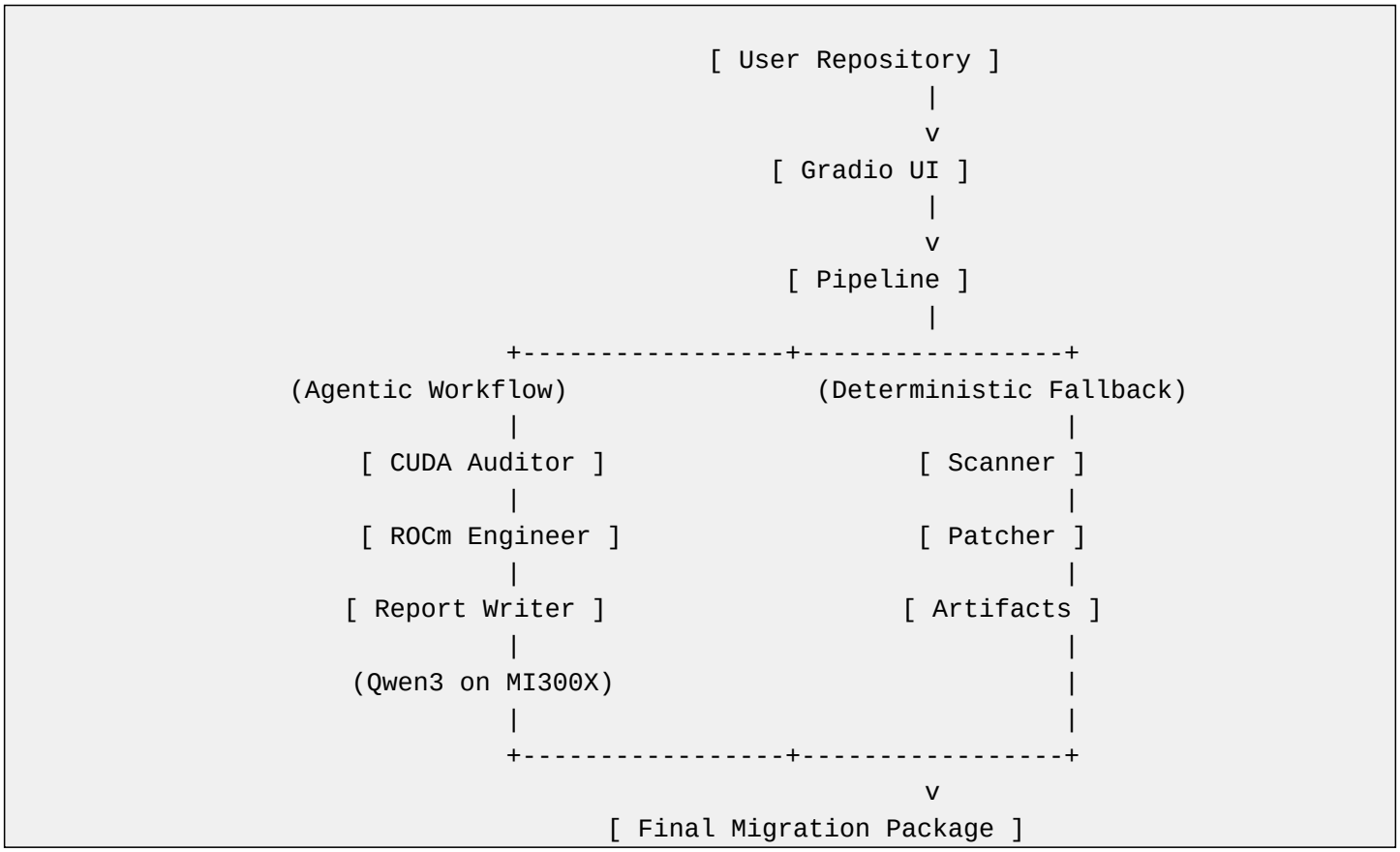
Millions of lines of legacy AI code are locked into CUDA.

Manually porting PyTorch code to ROCm is tedious, error-prone, and acts as a massive bottleneck for enterprises adopting AMD hardware.

The Goal:

Drop the switching cost to AMD infrastructure to ZERO using open-source AI agents.

2. System Architecture



3. Business Value & Implementation

ROCmPort AI completely automates the migration process using a 3-agent CrewAI pipeline:

- CUDA Auditor: Scans ASTs for blocking hardware-specific code.
- ROCm Engineer: Drafts the unified ROCm patch for PyTorch.
- Report Writer: Packages the diff alongside a generated Dockerfile and Runbook.

Business Impact:
Saves hundreds of developer hours per repository, allowing immediate utilization of high-performance AMD hardware without manual rewrites.

4. Hardware Proof (AMD MI300X)

We deployed the agent-generated patch directly onto the AMD Developer Cloud.

```
Hardware: AMD Instinct MI300X (192 GB HBM3)
ROCm: ROCm 7.0 (via Quick Start PyTorch container)
Model: Qwen/Qwen2.5-0.5B-Instruct
Throughput tokens/sec: 67.7
P50 latency ms: 1884.49
Peak VRAM GB: 2.05
```

Result: Verified successful migration! Original PyTorch code was executed natively on AMD MI300X using the ROCm software stack.

5. Live Deployment

GitHub Repository:

https://github.com/nawangdorjay/rocmport-ai

Hugging Face Demo:

https://huggingface.co/spaces/Nawangdorjay/rocmport-ai

YouTube Pitch:

https://youtu.be/3CDWDIOEwh0